09-24: lexical analysis (part 2)

In the previous part, we have explained the role of lexical analysis in the compilation process: the input text is turned (divided) into a sequence of words, so-called lexical symbols (or tokens). The text is being read from left to right, character by character, and the lexical analyser (scanner, lexer) is assessing if any of the symbol classes (regular sub-languages) includes the current prefix. Furthermore, if there are multiple prefixes that could be matched, the scanner ensures that the longest will be chosen (principle of maximal munch) and if the longest match belongs to multiple lexical categories, the higher priority will be selected (principle of token priority).

The language inclusion check is implemented with DFAs (deterministic finite automaton) generated from regular expressions defining the lexical categories. We use Thompson's construction to get an NFA (non-deterministic finite automaton) from the regular expression, then eliminate epsilon-transitions and non-deterministic choices, yielding a DFA. In general, there are various lexical categories in a language, so we have many such DFAs, each defining a regular language. As discussed in the previous part, we can implement a naive scanner by running all these DFAs on the text being read and checking acceptance; see the first scanner example that implements both principles over a language with three categories.

Constructing a single DFA

Having created a naive scanner, we understand the working principles, but we also see that there are various ways to improve the approach. Foremost, the lexical categories may overlap, or at least may share prefixes, which essentially results in repeating the "same job" when running the "similar" automata states belonging to the different categories. We overcome this limitation by merging the said states: common prefixes recognized by the various categories will be processed by a common part, we merge the different categories automata into a single DFA.

As one would expect, in case the categories share common prefixes, taking the union (of expressions, automata) results in nondeterministic choices. However, after constructing a single, joint NFA, we can use the previously discussed subset construction algorithm to eliminate nondeterminism by making common states by merging subsets of states into single states. See the second page in **this figure (https://canvas.elte.hu/courses/66503/files/4518841/download?wrap=1)** for our example that we discussed step by step in the lecture (note that we do not include an explicit error/trap state in the figure, but any transitions not drawn in the figure are implicitly leading to an error state). Note also that in the unified automaton, the accepting states are tagged with the lexical category they accept, and if a state was accepting in multiple components, priority is considered. The implementation must respect this.

The single, category-aware DFA can be implemented similarly to the simpler DFAs we discussed earlier, the only major difference is that the corresponding function (carrying out the DFA execution) not only returns the length of the longest match but also tells which category it was recognized in. See the implementation in **this file (https://canvas.elte.hu/courses/66503/files/4518843/download?wrap=1)** . The interested reader is encouraged to try to rewrite the branching-based implementation to a table-driven

DFA execution and thus abstract away the definition of the DFA from its execution.

DFA minimization

It is discussed in the Cornell handout that the joint automaton (constructed from the union of the regular expressions of the lexical categories) is likely to be verbose, and it is subject to reduction. We do not discuss this aspect in detail, but highlight the fact that unreachable states shall be removed as well as indistinguishable states shall be merged before further steps. After this process, the DFA recognizing the tokens becomes more compact and therefore its execution can be more efficient.

Cutting at hopeless states

Further optimisation is possible by remembering hopeless states. The scanner, when recognizing a token that may be a prefix to a longer match, will read the rest of the input (potentially up to a finite bound) and after finding the longer match impossible, will fall back to the shorter one. Then, in a subsequent step, the scanner may encounter the same situation, reading the rest of the input again, running through the same states, and finding out the same conclusion. This may happen again and again, which makes the approach quadratic in the length of the input (see the implementation that showcases this problem **in this file (https://canvas.elte.hu/courses/66503/files/4518844/download?wrap=1)**).

A very effective optimisation to avoid this is memoizing the hopeless states, that is, the ones that cannot work out positively (i.e. with a successful match) at particular positions in the text. We introduce a set to store the hopeless position-state pairs, and whenever we find a match, we store the subsequently visited states in a stack, and upon failure to find a longest match, we store the hopeless states in the set. On the other hand, when running the automaton, before making a step, we keep checking if the current setup (state-position pair) is hopeless, and if so, we terminate the loop and accept the most recent match. The implementation can be found in **this file (https://canvas.elte.hu/courses/66503/files/4518845/download?wrap=1)** , please download it, try it out and play around with it --- you may want to add more token types and try different input texts to see how the scanner behaves.

- Cornell handout: **https://www.cs.cornell.edu/courses/cs4120/2023sp/notes/#lexing (https://www.cs.cornell.edu/courses/cs4120/2023sp/notes/#lexing)**
- Stanford handout: **https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/040%20Lexical%20Analysis.pdf (https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/040%20Lexical%20Analysis.pdf)**